

LAYERZERO · SECURITY REVIEW

LayerZero

Canton OFT Review.

A security review of Canton OFT Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
LayerZero	Canton OFT Review	Guardian	January 19 – January 28, 2026	February 5, 2026

§ 00

Contents.

86 pages — overview, confidence ranking, scope, 58 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 58 findings	P. 11
§ 05	Disclaimer	P. 85
§ 06	About Guardian	P. 86

§ 01

Overview.

Guardian reviewed Canton OFT Review across a single review round. The final report includes 2 Critical, 5 High, 15 Medium, 24 Low, 12 Informational findings.

AUDIT FIRM	METHODOLOGY
Guardian	Smart contract · Manual · Main Review
CLIENT	
LayerZero	
PROJECT	
Canton OFT Review	
PREPARED BY	
Guardian	
AUDIT WINDOW	
January 19 – January 28, 2026	
FINAL REPORT	
February 5, 2026	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	2	0	2	0	0
● High	5	0	5	0	0
● Medium	15	0	15	0	0
● Low	24	0	24	0	0
● Informational	12	0	12	0	0
All severities	58	0	58	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Canton OFT Review.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Message encoding length not enforced	Logical Error	CRITICAL	ACKNOWLEDGED	p. 11
C-02	LzSendRequest_Create can be called manually	Access Control	CRITICAL	ACKNOWLEDGED	p. 12
H-01	Execute deadlines not enforced	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 15
H-02	Single Manual Payout Action Locks The Rest	DoS	HIGH	ACKNOWLEDGED	p. 17
H-03	Stale OftFactory in LzSendRequest, causes DoS	DoS	HIGH	ACKNOWLEDGED	p. 18
H-04	Allocation escrow bypass via transfer rule	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 20
H-05	Sender can redirect fees via feeCid receiver	Validation	HIGH	ACKNOWLEDGED	p. 23
M-01	Shared decimal truncation burns dust	Rounding	MEDIUM	ACKNOWLEDGED	p. 25
M-02	Allocation visibility blocks settlement	Trust Assumptions	MEDIUM	ACKNOWLEDGED	p. 27
M-03	LzSendRequest expire needs disclosure	Trust Assumptions	MEDIUM	ACKNOWLEDGED	p. 30
M-04	Missing RequestType Validation In Committer	Validation	MEDIUM	ACKNOWLEDGED	p. 33
M-05	State Commitments Can Be Arbitrarily Ordered	Validation	MEDIUM	ACKNOWLEDGED	p. 34

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-06	Empty signatures accepted in StateCommitment	Signatures	MEDIUM	ACKNOWLEDGED	p. 35
M-07	IzSend fails when sender is a payout	DoS	MEDIUM	ACKNOWLEDGED	p. 36
M-08	Lock holders cannot unlock holdings	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 37
M-09	Blacklist bypass through pending contracts	Logical Error	MEDIUM	ACKNOWLEDGED	p. 38
M-10	Withdraw bypasses allocateBefore; mirrors Cancel	Validation	MEDIUM	ACKNOWLEDGED	p. 39
M-11	Integer Floor Division Causes Fee Precision Loss	Rounding	MEDIUM	ACKNOWLEDGED	p. 40
M-12	Fractional offer/allocation amounts	Logical Error	MEDIUM	ACKNOWLEDGED	p. 41
M-13	Commitments Are Not Tied To Requests Fund Them	Validation	MEDIUM	ACKNOWLEDGED	p. 42
M-14	Unsupported Token Amounts	Logical Error	MEDIUM	ACKNOWLEDGED	p. 43
M-15	Receiver Needs Transfer Rule Disclosure	Trust Assumptions	MEDIUM	ACKNOWLEDGED	p. 44
L-01	Observers Are Not Propagated In Commitment	Validation	LOW	ACKNOWLEDGED	p. 45
L-02	Cancelled Amulet Transfers Cause Locked Funds	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 46
L-03	TransferImpl bypasses blacklist	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 47
L-04	Non-hex compose payload accepted	Validation	LOW	ACKNOWLEDGED	p. 48
L-05	feeCid Reuse Can Lock Cross-Chain Requests	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 49
L-06	Update returns auth error not constant	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 51

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-07	Fee lock mismatch bricks request expiry	DoS	LOW	ACKNOWLEDGED	p. 52
L-08	Local decimals mismatch not enforced	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 53
L-09	Redundant Holding State Check During IzSend	Logical Error	LOW	ACKNOWLEDGED	p. 54
L-10	Empty lockers brick locked holdings	Validation	LOW	ACKNOWLEDGED	p. 55
L-11	Fee refund context crashes on missing keys	Error	LOW	ACKNOWLEDGED	p. 56
L-13	Missing requestedAt <= now in Allocation	Validation	LOW	ACKNOWLEDGED	p. 57
L-14	Missing Validations When Updating Factory Params	Validation	LOW	ACKNOWLEDGED	p. 58
L-15	Missing Replay Protection For OApp_LzReceive	Trust Assumptions	LOW	ACKNOWLEDGED	p. 59
L-16	Request factory accepts mismatched IOApp	DoS	LOW	ACKNOWLEDGED	p. 60
L-17	IzSend accepts negative dstEid values	Validation	LOW	ACKNOWLEDGED	p. 61
L-18	IzSend accepts unbounded extraOptions size	Validation	LOW	ACKNOWLEDGED	p. 62
L-19	No Cap or Refund On Fee Payments	Validation	LOW	ACKNOWLEDGED	p. 63
L-20	Expired fee not validated at request creation	Validation	LOW	ACKNOWLEDGED	p. 64
L-21	stateRoot accept arbitrary Text not only bytes32	Validation	LOW	ACKNOWLEDGED	p. 65
L-22	requestFactoryCid Becomes Stale on Upgrade	DoS	LOW	ACKNOWLEDGED	p. 66
L-23	AllocationFactory allocate needs disclosure	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 67

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-24	Missing Time Validations In Allocation Creation	Validation	LOW	ACKNOWLEDGED	p. 69
L-25	Missing Validations When Creating Offers	Validation	LOW	ACKNOWLEDGED	p. 70
I-01	Misleading Comment In createPayoutHoldings	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 71
I-02	Endpoint Can Exercise Callback Choices	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 72
I-03	Duplicate payout receivers not coalesced	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 73
I-04	Missing issuer auth on OApp LzSend	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 74
I-05	Cannot burn locked holdings in BurnMint	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 76
I-06	No validation on SetExpirableAfter	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 77
I-07	No validation on SetMinLockDuration	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 78
I-08	DoS Window on New Cross-Chain Sends	Upgradeability	INFORMATIONAL	ACKNOWLEDGED	p. 79
I-09	Unused Errors	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 81
I-10	Sender Visibility Inconsistency in Request Event	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 82
I-11	Transfer rule upgrade can brick pending requests	Upgradeability	INFORMATIONAL	ACKNOWLEDGED	p. 83
I-12	Developed testing suite	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 84

CRITICAL **C-01**

ACKNOWLEDGED

Message encoding length not enforced

CATEGORY Logical Error
LOCATION MessageEncoding.daml
REVIEW Main Review

DESCRIPTION

`encodeMessage` builds the OFT payload by padding `sendParam.to` to 64 hex chars and then appending `amountSD` and optional fields. The hex validation only checks characters and strips 0x, and `padLeft` does not truncate, so a caller can supply a to value longer than 32 bytes. The encoded message then places the overflow bytes before the amount field. The receiver decoder uses fixed offsets for `sendTo` and `amountSD` and only checks a minimum length, so it will read `amountSD` from those overflow bytes rather than from the intended amount field. This is validated against the endpoint TypeScript decoder, which reads `amountSD` at fixed offsets and only enforces a minimum message length. The decoder implementation is in `packages/endpoint/src/oapp-common/oft-message-codec.ts` and the minimum length check is in `packages/endpoint/src/oapp-common/oft-receiver.ts`. An attacker can therefore control the decoded amount independent of the amount burned or locked on the source, which breaks cross-chain accounting.

Impact: Cross-chain mint amounts can be attacker-controlled and exceed the locked amount, leading to inflation or theft on the destination chain.

RECOMMENDATION

Enforce canonical length and even-length hex for `sendParam.to` after stripping 0x, rejecting values longer than 64 hex chars and odd-length inputs. If 20-byte addresses are supported, explicitly left-pad to 32 bytes before encoding. Validate `composeMsg` as hex bytes or remove it from the on-ledger message encoding and add strict length checks on the receiver decoder so it requires exact lengths for the non-compose and compose layouts.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/2089ffe07c901016b45fd853892206f7>

RESOLUTION

Status: Acknowledged.

CRITICAL **C-02**

ACKNOWLEDGED

LzSendRequest_Create can be called manually

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/main/daml/Layerzero/IRequestFactory/daml/ILzRequestFactory.daml#L72>

REVIEW Main Review

DESCRIPTION

The `LzSendRequest_Create` choice allows users to bypass critical validation by calling it directly instead of going through the intended `OftFactory.lzSend` flow.

The choice has `controller sender, issuer`, meaning both parties must authorize. However, an attacker can set `issuer = sender` (themselves), satisfying the controller requirement without actually being the token issuer. The implementation does not validate that the `issuer` parameter matches the actual token issuer (`instrumentId.admin`).

```
LzSendRequest_CreateImpl ILzRequestFactory.LzSendRequest_Create{..} = do
  validateFee feeCid sender minFee minLockDuration
  create LzSendRequest.LzSendRequest with
    sender
    issuer -- Can be set to sender, bypassing authorization
    lzMessage -- Can be arbitrary, bypassing encoding validation
    -- ...
```

The normal flow calls:

- `MessageEncoding.encodeMessage` which:
 - Validates hex format of destination address (`assertValidHex`)
 - Converts amount to shared decimals
 - Properly formats the message according to LayerZero protocol
- `OftFactory.lzSend` calculates and pays issuer fees:
 - Calculates fee: `oftFeeAmount = (amountLD * issuerFeeBps) / bpsDenominator`
 - Creates payout holdings for the issuer
 - Creates transfer offers for the issuer
- Passes these as `payoutCids` to `LzSendRequest_Create`

C-02 LzSendRequest_Create can be called manually

CONTINUED

By calling `LzSendRequest_Create` directly, attackers can pass `payoutCids = []` (empty), completely bypassing issuer fee payments, and can craft arbitrary `LzMessage` values that skip all this validation. This allows an attacker to create an encoded message claiming a significant amount of tokens, craft the `LzMessage` themselves, and cause minting on the destination chain without actually locking the corresponding tokens on Canton.

Exploitation ScenarioActors

- LegitIssuer: The real token admin (`instrumentId.admin`), e.g., "USDC on Canton".
- Attacker: Owns some USDC holdings.
- Fakelssuer: A party controlled by the attacker (could be another wallet identity).
- LzEndpoint: The relayer / operator that watches `LzSendRequest` and mints/burns on the destination chain.

Preconditions

1. `LzRequestFactory` is disclosed to the user (required for normal `LzSend` flow).
 2. The attacker can sign as sender and issuer.
- They set issuer = Fakelssuer (which they control), so they can sign both.

Step-by-Step Exploit

1. Attacker owns a real holding of the legitimate token:
 - `oftHoldingCid` corresponds to `instrumentId.admin` = LegitIssuer
 - Example: USDC holding owned by attacker.
2. Attacker creates a request directly (bypassing `OftFactory`):
 - Calls `LzSendRequest_Create` on the request factory.
 - Supplies:
 - sender = Attacker
 - issuer = Fakelssuer (attacker-controlled)
 - `oftHoldingCid` = USDC holding (admin is LegitIssuer)
 - valid feeCid
 - This succeeds because the factory never checks that `issuer == holding.instrumentId.admin`.
3. Ledger now contains a request that claims:
 - issuer = Fakelssuer
 - but the locked holding is actually a LegitIssuer asset.
4. Off-chain relayer reads the request and uses issuer as the routing key:
 - This is common: issuer or admin is used to pick the destination token contract, fee schedule, allowlists, or compliance rules.
 - The relayer now believes this is a Fakelssuer token.

C-02 LzSendRequest_Create can be called manually

CONTINUED

5. Bad outcomes happen depending on off-chain logic:

- Wrong asset minted on the destination chain (mismatch between real token and issuer metadata).
- Compliance / allowlist bypass: relayer may only enforce issuer-based constraints.
- DoS / funds stuck: relayer might refuse to process if issuer isn't recognized, leaving user funds locked or burned.
- Accounting errors: issuer-based fee splitting or limits are applied incorrectly.

Additional Impact Example (Allocation Escrow Bypass)

This is a distinct sink-validation risk that becomes reachable when arbitrary `oftHoldingCid` can be supplied. Consider a DvP allocation: Alice allocates to Carol, and Bob is the executor. The allocation contract stores `lockedHoldingCid`, so Bob can see the escrow CID. Bob then creates a manual `LzSendRequest` referencing Alice's allocation escrow as `oftHoldingCid`. When the endpoint accepts, the callback burns/unlocks that allocation escrow even though it was not created for LzSend, bypassing the allocation settlement rules.

POC for this example: <https://gist.github.com/GuardianAudits/c85fa03962d1d126f3e2b057f33a5b00>

— RECOMMENDATION

Consider adding validation to ensure the `issuer` parameter matches the actual token issuer. Additionally, bind callback consumption to the specific `LzSendRequest` and validate that `oftHoldingCid` (and `payoutCids`) were created for that request: enforce LzSend lock context, expected lockers (admin), and sender/instrument matching before burn/unlock.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/8cdacc163520b376150c3a84ae0abe31>

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

Execute deadlines not enforced

CATEGORY	Unexpected Behavior
LOCATION	OftTransferOffer.daml, OftTransferRuledaml, OftFactory.daml, OftAllocation.daml, OftFactory.daml
REVIEW	Main Review

— DESCRIPTION

The transfer and allocation flows do not enforce execution time windows at the point of acceptance or settlement. Transfers only validate time ordering at creation, but the accept path executes without checking that the current ledger time is before `executeBefore`. Allocations only enforce `requestedAt < allocateBefore < settleBefore` at creation, but the `executeTransfer` path does not check `now <= settleBefore`. If callers treat these fields as enforceable deadlines, a receiver can accept a transfer after its `executeBefore` or an executor can settle a DvP leg after `settleBefore`, which violates sender intent and higher level scheduling or compliance assumptions.

Impact: late acceptance or settlement can be exploited opportunistically after market moves or used to bypass time based controls expected by application logic. For example:

- 15:55 Alice initiates a 1,000,000 USDC transfer to Bob with `executeBefore` = 16:00 as part of an OTC swap and expects it to expire if not accepted by the cutoff.
- 16:05 USDT depegs to \$0.97; Bob now wants the USDC only after the move.
- Bob accepts at 16:10; `transferInstruction_acceptImpl` in `daml/Layerzero/Oft/daml/Transfer/OftTransferOffer.daml` doesn't check the deadline, so the ledger still completes the transfer.
- Outcome: Bob gets a free option (accept only when favorable), Alice is forced into an off-market settlement and time-based compliance controls are bypassed.

— RECOMMENDATION

Add ledger time checks in the accept and execute paths. In the transfer flow, enforce `now <= transfer.executeBefore` and optionally `transfer.requestedAt <= now` before executing the rule, either in the offer accept implementation or inside the transfer rule. In the allocation flow, enforce `now <= allocation.settlement.settleBefore` in `allocation_executeTransferImpl`. Decide desired semantics on expiry, such as reject with a specific error or auto fail and unlock the holding, and implement consistently.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/836bd6279d853cfc3a37dfa6e97722d0>

H-01 Execute deadlines not enforced

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Single Manual Payout Action Locks The Rest

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/c614e08a5acc017d755dbf0866854e8545b4d34e/daml/Layerzero/Oft/daml/OftFactory.daml#L220
REVIEW	Main Review

DESCRIPTION

Issuer can always exercise `ILockedPayout_ForceAccept` and `ForceReject` directly (e.g., tests `testLockHolderCanForceAccept/Reject` already do this), and nothing prevents early settlement of any payout offer.

`LzSendRequest` stores the entire `payoutCid`s list when the send is created, and the endpoint later hands that list to `OftFactory` during Accept/Reject/Expire. The endpoint cannot remove or replace entries even if they're no longer valid.

In each callback `OftFactory` loops `forA_ payoutCids (exercise ... ILockedPayout_Force*)` without checking whether the payout contract still exists. If the issuer already force-settled any entry, that exercise fails due to contract being archived.

Because the callbacks are atomic, the first archived payout aborts the entire flow: the cross-chain sending holding never burns/unlocks, remaining payouts never settle, and even expiry refunds get stuck.

Until the code tolerates missing CIDs (or forbids manual exercises), a single manual payout action by the issuer can block every other payout and the overall request.

RECOMMENDATION

Wrap the `ILockedPayout_Force*` loop in a safe lookup so stale CIDs are skipped instead of aborting the entire cross-chain flow or block manual settlements.

PROOF OF CONCEPT <https://gist.github.com/osmanozdemir1/63bce9139ba8466b448b6ff0f8a2cb17>

RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

Stale OftFactory in LzSendRequest, causes DoS

CATEGORY	DoS
LOCATION	LzSendRequest.daml
REVIEW	Main Review

DESCRIPTION

In Daml, when a contract is updated using a consuming choice that returns a new `ContractId`, the original contract is archived and a new contract with a different `ContractId` is created. The `LzSendRequest` template stores a `ContractId IOApp` reference to the factory, but this reference becomes invalid if the factory contract is updated.

When the `OftFactory` contract is updated via admin choices (e.g., `SetIssuerFeeBps`, `SetRequestFactory`, `AddToBlacklist`, etc.):

```
-- | Update the issuer fee in basis points
choice SetIssuerFeeBps : ContractId OftFactory
with
  newIssuerFeeBps : Int
  controller instrumentId.admin
do
  assertMsg "ERR_INVALID_FEE_BPS" $ newIssuerFeeBps >= 0 && newIssuerFeeBps <= 10000
  create this with issuerFeeBps = newIssuerFeeBps
```

This choice archives the old factory and creates a new one with the updated fee.

The problem is that if a `LzSendRequest` is pending, was initiated before the update, the stored `ContractId` in the request becomes stale:

```
-- Create the LzSendRequest contract
create LzSendRequest.LzSendRequest with
  sender
  issuer
  lzEndpoint = owner
  oftHoldingCid
  dstEid
  lzMessage
  extraOptions
```

H-03 Stale OftFactory in LzSendRequest, causes DoS

CONTINUED

```
feeCid
hasCompose
composeMsg
expirableAfter = lzSendRequestExpirableAfter
oftFactoryCid
payoutCids
```

This causes callback exercises to fail because they attempt to exercise choices (callbacks) on an archived contract:

```
-- Callback to OftFactory (via OApp interface) to burn the holding and force-accept payouts
exercise oftFactoryCid OApp_OnLzSendAcceptedCallback with
oftHoldingCid
payoutCids
```

— RECOMMENDATION

Consider replacing stored `oftFactoryCid` in `LzSendRequest` with `instrumentId` and resolving the current `OftFactory` at callback time via a contract key (`fetchByKey`). This avoids stale `ContractId` failures after consuming updates (e.g., `SetIssuerFeeBps` , `AddToBlacklist`) and keeps pending requests functional across factory rotations.

```
-- OftFactory
key instrumentId : Holding.InstrumentId
maintainer key.admin
```

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/a5c5af6a0d11f1ce8a71b325cf8ac244>

— RESOLUTION

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

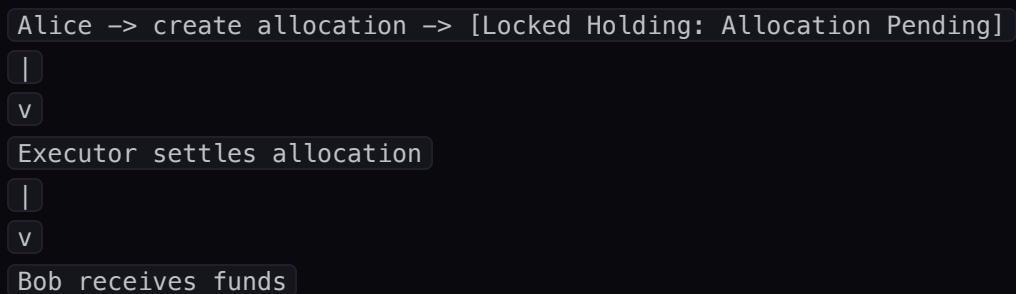
Allocation escrow bypass via transfer rule

CATEGORY	Unexpected Behavior
LOCATION	OftTransferRule.daml, Utils.daml
REVIEW	Main Review

DESCRIPTION

Allocations are meant to implement delivery versus payment style settlement flows where funds are reserved in escrow while a coordinator or executor orchestrates the final settlement. The purpose is to lock the sender's holdings so the assets cannot be moved through ordinary transfer paths until the allocation is either settled, canceled or withdrawn according to the allocation rules. In other words, the allocation lock is supposed to be the boundary that guarantees atomicity, approvals and any settlement-time policies such as fees or checks enforced by the settlement workflow.

The intended flow is that a sender creates an allocation, the holding is locked under an allocation specific context, and only the allocation settlement path releases or transfers the locked holding. A simplified view of the intended flow is:



In the current implementation, that escrow boundary can be bypassed because the normal transfer rule accepts any locked holding without checking why it is locked or who is allowed to unlock it. The transfer rule simply verifies that the input holding is locked and owned by the sender, then unlocks it and transfers it. This means an allocation locked holding can be spent directly via the standard transfer rule, even though it was created for a different workflow.

A concrete exploitation path looks like this. First, Alice creates an allocation to Bob for a fixed amount, which creates a locked holding that is marked as allocation pending. That locked holding is intended to be released only when the executor settles the allocation or cancels it according to the allocation rules. Next, before any executor action happens, Alice and Bob submit a normal transfer that uses that locked holding as input. The

H-04 Allocation escrow bypass via transfer rule

CONTINUED

transfer rule accepts the locked holding, unlocks it and transfers the funds to Bob immediately. The allocation contract is now stranded because its locked holding has been consumed and the escrow guarantee has been bypassed.

This bypass also skips any allocation specific enforcement that would normally happen during settlement, such as coordinator approval, time window checks, or fees that a settlement service charges during allocation execution. Because the transfer rule path never invokes the allocation settlement logic, any fee or approval that is applied there is effectively avoided. The impact is that escrowed funds can be moved early, allocation workflows can be sidestepped, and any policies that rely on the allocation settlement step are no longer enforced. This breaks settlement integrity and can violate compliance or commercial expectations.

The same design weakness can also affect other escrowed holdings that use different lock contexts, such as the `LzSend` flow. A holding locked with context "LzSend pending" can be unlocked and transferred through the generic transfer rule, leaving a pending `LzSendRequest` that cannot be finalized because its locked holding was consumed. This is not a theft by a third party, but it does allow the sender and receiver to bypass the intended cross chain escrow path and can brick the request and fee settlement.

— RECOMMENDATION

Reject allocation locked holdings in the transfer rule or enforce a lock context whitelist so only transfer related locks can be unlocked by the transfer rule. Another safe option is to add context or lock holder validation inside the shared holding action helper so `UnlockMergeSplitTransfer` only succeeds for locks intended to be released by that path. One concrete fix is to gate `UnlockMergeSplitTransfer` in the transfer rule based on the lock context, for example:

```
let lockedOftCids = map fromInterfaceContractId transfer.inputHoldingCids
forA_ lockedOftCids $ \cid -> do
  h <- fetch cid
  case h.lock of
  Some l ->
    assertMsg errMsgTransfer_LockContextNotAllowed $
      isTransferLockContext l.context
  None -> assertMsg errMsgTransfer_ExpectedLocked False
  result <- applyHoldingAction
    (UnlockMergeSplitTransfer transfer.receiver)
    transfer.sender
    instrumentId
```

H-04 Allocation escrow bypass via transfer rule

CONTINUED

`lockedOftCids``transferAmount`

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/01acfdbacb36127a6e5655645518572a>

— RESOLUTION

Status: Acknowledged.

HIGH **H-05**

ACKNOWLEDGED

Sender can redirect fees via feeCid receiver

CATEGORY Validation
 LOCATION FeeUtils.daml
 REVIEW Main Review

DESCRIPTION

The `validateFee` function in `FeeUtils.daml` does not check that the fee instruction receiver is `lzEndpoint`.

In Daml, when a contract exercises a choice on another contract, the authorization set is built from:

1. The controllers of the exercised choice.
2. The signatories of the contract on which the choice is exercised (automatically included for all nested exercises).

```
LzSendRequest :
template LzSendRequest
...
where
signatory sender, lzEndpoint

AmuletTransferInstruction :
template AmuletTransferInstruction
...
where
signatory transfer.instrumentId.admin, transfer.sender
```

The `TransferInstruction_Accept` choice is controlled by `transfer.receiver`.

When `lzEndpoint` accepts an `LzSendRequest`, the flow is:

1. Top-level submission:

```
submit (actAs lzEndpoint <> disclose ...) do
exerciseCmd lzSendRequestCid ILzSendRequest_Accept with amuletContext
```

Authorization set: `{lzEndpoint}`.

1. Exercising `lzSendRequest_AcceptImpl`:

H-05 Sender can redirect fees via feeCid receiver

CONTINUED

```
lzSendRequest_AcceptImpl ... = do
  acceptFee feeCid amuletContext
```

The authorization set now includes:

- {lzEndpoint} from actAs
- {sender, lzEndpoint} from LzSendRequest signatories

Final set: {sender, lzEndpoint}.

1. Exercising acceptFee:

```
acceptFee feeCid amuletContext = do
  exercise (toInterfaceContractId @TransferInstruction.TransferInstruction feeCid)
  TransferInstruction.TransferInstruction_Accept ...
```

Since TransferInstruction_Accept is controlled by transfer.receiver, and sender is already in the authorization set (due to being a signatory of LzSendRequest), this exercise succeeds even if the receiver is not lzEndpoint.

As a result, a sender can create a fee instruction where the receiver is themselves. When lzEndpoint accepts the LzSendRequest, the fee is accepted and transferred to the sender instead of lzEndpoint, even though lzEndpoint is the only party that explicitly submitted the transaction.

This is caused by:

- Missing validation that fee.receiver == lzEndpoint.
- Daml's authorization model automatically extending the authorization set with signatories during nested exercises.

Together, they allow fee redirection.

— RECOMMENDATION

Consider adding a validation to the validateFee function to check that the fee instruction's receiver is lzEndpoint.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/f6af5875ee6d082ab3df83a2da5c97e0>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Shared decimal truncation burns dust

CATEGORY Rounding
LOCATION MessageEncoding.daml
REVIEW Main Review

DESCRIPTION

The cross-chain send path converts a local unit amount into shared decimals using integer division and then encodes that truncated shared amount in the message. There is no on-ledger check that the local amount is divisible by the conversion rate and no refund of the remainder, so any fractional part is silently discarded. The locked and later burned local holding reflects the full local amount, while the payload only represents the truncated shared amount. The receive-side decoder derives the shared amount from the payload and passes it forward as the receive amount, and the on-ledger receive mints exactly the amount it is given. This means the destination side only receives the truncated shared amount while the source burns the full local amount, leaving the remainder unrecoverable as dust.

The truncation happens in the conversion helper:

```
toSharedDecimals : Int -> Int -> Int -> Int
toSharedDecimals amount localDecimals sharedDecimals =
  let decimalConversionRate = localDecimals - sharedDecimals
  in amount / 10 ^ decimalConversionRate
```

The send flow locks the full local amount and then encodes the truncated shared amount:

```
result <- applyHoldingAction (MergeSplitLock lzSendLock) sender instrumentId [oftCid]
oftAmountToSendCrossChain
let (encodedMessage, hasCompose) = encodeMessage sender oftAmountToSendCrossChain
localDecimals sharedDecimals sendParam
```

The receiver extracts `amountSD` from the payload and forwards it as the receive amount:

```
const amount = oftMessageCodec.amountSD(msgBytes);
const instruction = { type: 'lzReceive', toAddress: receiver, amount };
context.addInstruction(instruction);
```

The on-ledger receive mints exactly that amount:

```
lzReceive instrumentId localDecimals receiver amount = do
  create Oft with
```

M-01 Shared decimal truncation burns dust

CONTINUED

```
owner = receiver
amount
instrumentId
localDecimals
lock = None
```

As a concrete example with `localDecimals = 8` and `sharedDecimals = 6`, a user sending 105 local units will have `amountSD = 1` and the message represents 100 local units, while the locked holding is still 105. The receive-side amount is 1, so only the equivalent of 100 local units is represented on the destination and the remaining 5 are burned with no refund. For amounts smaller than the conversion rate, `amountSD` becomes 0 and the receive-side mint fails because holdings require `amount > 0`, while the source may already have burned the full local amount. This is demonstrated in the PoC, which shows the encoded message contains the truncated shared amount while the locked holding remains the full local amount.

Finally, do notice that the `minAmountLD` check does not guard against truncation because it compares local amounts before conversion, so it can still pass while the encoded shared amount drops below expectations.

— RECOMMENDATION

Enforce a canonical conversion boundary in the send flow. Compute the conversion rate, then either reject any amount with a nonzero remainder or split the holding so only the divisible portion is locked and burned while the remainder is returned to the sender as a new unlocked holding. Ensure the encoded shared amount always corresponds exactly to the local amount that will be burned or minted on the destination.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/380b21c7344cdaf6d145b64593aac7f0>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Allocation visibility blocks settlement

CATEGORY Trust Assumptions
 LOCATION AllocationImpl.daml
 REVIEW Main Review

DESCRIPTION

In the allocation flow, the locked holding created for the DvP leg does not add any observers, and the allocation contract itself only lists the executor as an observer. Under Canton privacy, a party must see a contract to authorize a choice on it, so the receiver and executor may not be able to authorize settlement or cancellation unless they are explicitly disclosed both the allocation and the locked holding. This creates a liveness dependency on disclosure infrastructure rather than the on-ledger contracts.

The allocation lock is created with no observers, which means only the issuer (signatory) and the sender (owner) see the locked holding by default, while the receiver and executor do not.

```
lockers = [instrumentId.admin]
context = "Allocation " <> show allocation.settlement.settlementRef <> ";" <> show
allocation.transferLegId
observers = None
```

The allocation contract itself makes only the executor an observer, while the receiver is not. That is incompatible with the interface requirement that the receiver must authorize settlement or cancellation.

```
template OftAllocation
with
allocation : AllocationV1.AllocationSpecification
lockedHoldingCid : ContractId Oft
where
signatory allocation.transferLeg.instrumentId.admin, allocation.transferLeg.sender
observer allocation.settlement.executor
```

The interface requires joint authorization by executor, sender, and receiver for both execute and cancel. If the receiver cannot see the allocation, they cannot co-authorize these choices, and the transaction fails even when all parties intend to settle.

M-02 Allocation visibility blocks settlement

CONTINUED

```
allocationControllers AllocationView{..} =
[allocation.settlement.executor, allocation.transferLeg.sender,
allocation.transferLeg.receiver]
choice Allocation_ExecuteTransfer : Allocation_ExecuteTransferResult
controller allocationControllers (view this)
do allocation_executeTransferImpl this self arg
choice Allocation_Cancel : Allocation_CancelResult
controller allocationControllers (view this)
do allocation_cancelImpl this self arg
```

The test probe shows the receiver cannot see or exercise the allocation without disclosure, confirming the dependency on off-ledger disclosure.

```
None <- queryContractId bob (fromInterfaceContractId @OftAllocation allocationCid)
submitMustFail (actAs bob) do
exerciseCmd allocationCid Allocation.Allocation_ExecuteTransfer with
extraArgs = ExtraArgs with context = ChoiceContext TM.empty; meta = emptyMetadata
```

This creates a production liveness risk. If disclosure infrastructure is down or delayed, allocations can neither settle nor cancel. Locked funds remain in escrow until the sender withdraws or the issuer intervenes, which can stall DvP settlements and strand assets during outages.

— RECOMMENDATION

Include the receiver and executor as observers on both the allocation contract and the allocation lock so they can see the allocation and locked holding by default and co-authorize settlement choices. A minimal change is to add those observers in the allocation and lock creation logic.

```
observer allocation.settlement.executor, allocation.transferLeg.receiver
let allocationLock = Lock with
lockers = [instrumentId.admin]
context = "Allocation " <> show allocation.settlement.settlementRef <> ";" <> show
allocation.transferLegId
observers = Some [allocation.transferLeg.receiver, allocation.settlement.executor]
```

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/8cbab40337aecc73d972b7310278385e>

M-02 Allocation visibility blocks settlement

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

LzSendRequest expire needs disclosure

CATEGORY Trust Assumptions
LOCATION LzRequestFactory.daml
REVIEW Main Review

DESCRIPTION

The sender-driven expire flow depends on contracts that the sender does not see by default. The `LzSendRequest_Expire` choice is exercised on the request factory interface, but the request factory contract is owned by the endpoint party, so it is not visible to the sender unless the sender attaches an explicit disclosure payload.

```
nonconsuming choice LzSendRequest_Expire : ()  
with  
sender : Party  
lzSendRequestCid : ContractId ILzSendRequest  
amuletContext : ChoiceContext  
controller sender  
do  
  lzSendRequest_ExpireImpl this arg
```

The factory implementation immediately exercises the `ILzSendRequest_Expire` choice on the referenced request contract. This requires the request contract to be visible or explicitly disclosed to the submitter.

```
lzSendRequest_ExpireImpl ILzRequestFactory.LzSendRequest_Expire{..} = do  
  exercise lzSendRequestCid ILzSendRequest_Expire with  
  amuletContext
```

Inside `LzSendRequest`, the expire path withdraws the fee and then calls back into the OFT `I0App` to unlock the locked holding and reject payouts. This introduces dependencies on the fee instruction and amulet context, plus the `I0App` contract referenced by `oftFactoryCid`.

```
lzSendRequest_ExpireImpl ILzSendRequest_Expire{..} = do  
  now <- getTime  
  assertMsg errLzSendRequest_ExpireTooEarly $ now >= expirableAfter  
  withdrawFee feeCid amuletContext  
  create LzSendRequestExpiredEvent with
```

M-03 LzSendRequest expire needs disclosure

CONTINUED

```

lzEndpoint
dstEid
lzMessage
extraOptions
feeCid
oftHoldingCid
hasCompose
composeMsg
sender
exercise oftFactoryCid OApp_OnLzSendExpiredCallback with
oftHoldingCid
payoutCids

withdrawFee feeCid amuletContext = do
_ <- exercise (toInterfaceContractId @TransferInstruction.TransferInstruction feeCid)
TransferInstruction.TransferInstruction_Withdraw with
extraArgs = ExtraArgs with
context = amuletContext
meta = emptyMetadata
pure ()

```

The tests already reflect this dependency by attaching explicit disclosures for the request factory, OFT factory, request contract, and amulet infrastructure before expiring, which means the flow assumes an off-ledger disclosure service is operational.

```

submit (actAs alice <> disclose factoryDisclosure <> disclose requestFactoryDisclosure'
<> disclose amuletRulesDisclosure <> disclose openRoundDisclosure
<> disclose lockedAmuletDisclosure <> disclose lzSendRequestDisclosure) do
exerciseCmd requestFactoryCid LzSendRequest_Expire with
sender = alice
lzSendRequestCid = toInterfaceContractId @ILzSendRequest lzSendRequestCid
amuletContext

```

If disclosure delivery is delayed or unavailable, the expire transaction fails with visibility errors and the sender cannot unlock the locked holding or withdraw the fee. This can strand funds until the endpoint acts, creating an availability risk tied to off-ledger disclosure plumbing rather than on-ledger observers.

— RECOMMENDATION

Make the expire path independent of off-ledger disclosure by adding the sender (and any party expected to exercise expire) as an observer to the request factory and the `IOApp`

M-03 LzSendRequest expire needs disclosure

CONTINUED

contract, or by adding a dedicated on-ledger delegation contract that grants the sender the ability to exercise expire without explicit disclosure. If explicit disclosure remains the intended approach, document it as an operational requirement and add runtime checks plus monitoring to detect disclosure failures before requests accumulate and funds are locked.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Missing RequestType Validation In Committer

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/c614e08a5acc017d755dbf0866854e8545b4d34e/daml/Layerzero/StateTransition/daml/Committer.daml#L65>

REVIEW Main Review

DESCRIPTION

`Request_Create` records `requestType` and `payload`, but `Request_Accept` and `Request_Reject` blindly emit whatever `requestType`, `acceptPayload`, `rejectPayload` the caller supplies. No fetch/assert ensures these values match the contract being consumed.

`AcceptRequest` and `RejectRequest` in `Committer` take the `requestType` and `payload` parameters from the caller and forward directly without checking whether it's the same type as the original request. Anyone with committer privileges can falsify the `RequestAcceptedEvent/RequestRejectedEvent` metadata and mislabel which action was processed, making it impossible to trust `requestType` or `payload` in downstream monitoring.

Similarly, there is no check when the sender performs `Request_Expire` on an expired request, and can provide any value.

RECOMMENDATION

When accepting/rejecting, fetch the target `Request` contract and assert `request.requestType == requestType` (and optionally that the payload matches) before exercising, or eliminate the caller-supplied fields and derive them directly from the stored contract to guarantee consistency.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

State Commitments Can Be Arbitrarily Ordered

CATEGORY Validation
LOCATION Committer.daml
REVIEW Main Review

— DESCRIPTION

Committer's `CreateStateCommitment` only fetches the supplied `prevStateCommitmentContractId` to confirm the contract exists but it never records or compares against a head before creating a new commitment.

The template claims this field represents "Previous state (None for genesis)" but nothing prevents the owner from pointing to any historical commitment—or even None—when creating fresh state. Since `AcceptRequest` and `RejectRequest` merely call `CreateStateCommitment`, the endpoint can create arbitrarily ordered commitments, introduce forks, or replay stale roots. Downstream readers cannot rely on these contracts to form a single append-only chain of verified state roots since there is no guarantee that `prevStateCommitmentContractId` is actually the previous state commitment.

— RECOMMENDATION

Track the latest commitment on the Committer contract, and require every new `StateCommitment` to reference that previous one before updating it. This would require the Committer contract to be archived and re-created with the updated commitment and ensures each new commitment points to the previous one.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/ddc26ce87d8862ba042d8d115e7765ba>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Empty signatures accepted in StateCommitment

CATEGORY Signatures
LOCATION StateCommitment.daml
REVIEW Main Review

DESCRIPTION

`StateCommitment` validation accepts a commitment even when the signatures list is empty. This means a commitment can be created and processed without any cryptographic approvals, even though the protocol intent is that signers attest to the payload and authorize the state transition. When downstream logic treats the presence of a `StateCommitment` as sufficient, an actor who can submit or relay a commitment can advance state without the intended quorum or any signer participation.

This breaks the security model that commitments represent authenticated, signed state updates.

Impact: unauthorized state transitions and the potential for inconsistent or forked state acceptance, which can undermine trust and lead to incorrect state or financial outcomes.

RECOMMENDATION

Reject commitments with an empty signatures list at the earliest validation boundary. Enforce a minimum signature count and verify each signature against the required signer set and threshold before accepting or applying the commitment.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

IzSend fails when sender is a payout

CATEGORY DoS
LOCATION 0AppImpl.daml
REVIEW Main Review

— DESCRIPTION

The cross chain send flow creates payout transfer offers for each receiver. Those offers use the standard transfer offer template, which enforces a strict invariant that the sender and receiver must be different. When the payout receiver list includes the sender, the offer creation fails and the entire IzSend transaction aborts. This is a hard failure, not a graceful rejection, and it occurs after the flow has already validated amounts and split holdings, so the caller just sees a failed send with no clear intent level error.

Example scenario: Alice is both the sender and an eligible payout receiver. This can happen if the issuer fee is enabled and the issuer is the sender, or if Alice is a service provider who receives a rebate. Alice submits a valid `IzSend` with a payout list that includes herself. The system attempts to create a payout offer from Alice to Alice, which violates the self transfer invariant and the transaction fails. In practice this allows a misconfiguration or crafted payout list to reliably block `IzSend` operations for a sender.

Impact: `IzSend` is unavailable in realistic configurations where the sender can also be a fee recipient, resulting in denial of service for cross chain transfers and operational disruption.

— RECOMMENDATION

Handle sender payouts explicitly before creating offers. If the sender appears in the payout list, net that amount into the sender refund or change holding instead of creating a self transfer offer. If the intended policy is to disallow sender payouts, reject early with a clear error before any holding operations.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

Lock holders cannot unlock holdings

CATEGORY Unexpected Behavior
LOCATION 0ft.daml
REVIEW Main Review

DESCRIPTION

The lock design allows a holding to be locked by one set of parties but only exposes the holding to the owner and optional lock observers. The unlock choice is controlled by the lock holders, yet those parties are not automatically observers. If a lock is created with lockers that are not also observers, the authorized lockers cannot see the holding to exercise the unlock choice. The result is a locked holding that no authorized party can unlock.

Example scenario: Alice receives a holding locked for an external executor Bob. The lock lists Bob as the sole locker and observers is left empty. Bob is the only party authorized to unlock, but Bob cannot see the holding because he is not an observer or signatory. The holding stays locked indefinitely, even though the system believes it is unlockable. This can be triggered by misconfiguration or by any workflow that uses non admin lockers without explicitly adding them as observers.

This is most likely to occur when an operator uses `0ft_Lock` directly for manual or operational holds and sets lockers to a third party without also adding them as observers. It also applies to any custom or future flow that reuses the lock type and assigns non admin lockers. The built in flows that create locks in this codebase set lockers to the instrument admin, so they do not hit the visibility gap, but any deviation from that pattern will.

Impact: holdings can be permanently stuck and unusable, breaking transfers and settlement flows that rely on external lockers.

RECOMMENDATION

Ensure lock holders always have visibility. Add `lockHolders` as observers on the holding, or enforce a rule at lock creation that the lockers set is included in observers. If visibility cannot be guaranteed, reject the lock creation to avoid creating un-unlockable holdings.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

Blacklist bypass through pending contracts

CATEGORY Logical Error
LOCATION OftTransferRule.daml
REVIEW Main Review

DESCRIPTION

The blacklist mechanism is enforced only at action initiation time (when creating a transfer, allocation, or cross-chain send), but it is not re-applied during execution of already created offers. In particular, blacklist checks are not performed when accepting or finalizing pending operations, allowing blacklisted parties to still receive tokens if the offer was created before blacklisting.

`assertNotBlacklisted` is applied in `OftFactory` entry points such as:

`TransferFactory_Transfer`
`AllocationFactory_Allocate`
`BurnMintFactory_BurnMint`
`OApp_LzSend`
`OApp_LzReceive`

However, execution paths such as `TransferInstruction_Accept` and related settlement logic do not re-validate the blacklist.

RECOMMENDATION

Enforce blacklist checks at execution time, not only at initiation time.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

Withdraw bypasses allocateBefore; mirrors Cancel

CATEGORY Validation
LOCATION OftAllocation.daml
REVIEW Main Review

DESCRIPTION

`allocation_withdrawImpl` lets the sender unlock and reclaim the allocated holding without enforcing `allocation.settlement.allocateBefore`. As a result, the sender can withdraw even after the allocation window has closed, when the protocol expects the allocation to be either executed or jointly cancelled.

Practically, the current `withdraw` implementation is equivalent to `cancel`: both choices just `Oft_Unlock` the same locked holding and return it to the sender.

This contradicts the `Allocation_Withdraw` specification, which states that withdrawal "SHOULD not fail settlement if the sender still has time to allocate again; i.e., the `settlement.allocateBefore` deadline has not yet passed."

https://docs.sync.global/app_dev/api/splice-api-token-allocation-v1/Splice-API-Token-AllocationV1.html#type-splice-api-token-allocationv1-allocationwithdraw-60458

Currently, the sender can unilaterally withdraw an allocation after `allocateBefore`, breaking the settlement flow when re-allocation is no longer possible.

RECOMMENDATION

Consider making `Withdraw` time-bounded and keep `Cancel` as the coordinated abort path. Add a ledger-time check:

```
now <- getTime
assertMsg "Withdrawal not allowed after allocateBefore deadline"
(now < allocation.settlement.allocateBefore)
```

RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Integer Floor Division Causes Fee Precision Loss

CATEGORY Rounding

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/main/daml/Layerzero/Oft/daml/OApp/OAppImpl.daml#L148>

REVIEW Main Review

DESCRIPTION

The fee calculation uses integer floor division, which truncates fractional amounts instead of rounding them up.

```
let oftFeeAmount = (amountLD * issuerFeeBps) / bpsDenominator
```

Bug Scenario:

- localDecimals = 0 → token has no fractional units
- amount = 19 tokens
- issuerFeeBps = 500 (5%)

$$\begin{aligned}\text{Fee} &= (\text{amountLD} \times \text{issuerFeeBps}) / 10,000 \\ &= (19 \times 500) / 10,000 \\ &= 9,500 / 10,000 \\ &= 0.95 \rightarrow 0 \text{ (integer floor)}\end{aligned}$$

So the fee collected will be 0. Loss 100% of the intended fee.

RECOMMENDATION

Use ceiling division instead of floor division to ensure fees are always rounded up to favor the protocol.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-12**

ACKNOWLEDGED

Fractional offer/allocation amounts

CATEGORY	Logical Error
LOCATION	TransferImpl.daml, AllocationImpl.daml
REVIEW	Main Review

— DESCRIPTION

Transfer and allocation amounts are `Decimal` but are truncated to `Int` without validating they are whole-number amounts. The offer/allocation record preserves the original decimal (e.g., `10.9`), but execution uses `truncate` (i.e receiver gets `10`).

This is a silent underpayment risk and can confuse UIs or off-chain accounting. A malicious user can also craft DvP allocations with fractional amounts to make a counterparty settle on the displayed value while actually receiving less.

— RECOMMENDATION

Reject non-integer `Decimal` amounts at the boundary (e.g., assert `amount == intToDecimal (truncate amount)`), or round explicitly and document the behavior to all clients.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-13**

ACKNOWLEDGED

Commitments Are Not Tied To Requests Fund Them

CATEGORY Validation
LOCATION Committer.daml
REVIEW Main Review

DESCRIPTION

When `Committer.AcceptRequest` or `RejectRequest` finishes processing a Request, it immediately calls `CreateStateCommitment` with whatever `stateRoot`, `tx`, and signatures the caller supplied. The `StateCommitment` template stores only those raw fields plus the optional previous commitment but it never records which `requestCid` produced the update.

Consequently, there is no on-ledger evidence linking a fee-paying request to the state commitment that supposedly resulted from it. The committer can accept request A (consuming its fee) but publish the state root/transaction for request B, or even fabricate entirely new data, and downstream observers have no way to detect the mismatch because the commitment lacks any reference back to the originating request.

RECOMMENDATION

Store the `requestCid` (and optionally its `requestType/payload hash`) inside `StateCommitment` and have `Committer` populate it directly from the exercised Request, so every commitment can be traced back to the exact request that funded it.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-14**

ACKNOWLEDGED

Unsupported Token Amounts

CATEGORY Logical Error
LOCATION GLOBAL
REVIEW Main Review

DESCRIPTION

OFT balances, transfers and allocation amounts are stored as `Int`. The OFT ensures token decimals are positive but it can support any value, like 18-decimal tokens. Even modest human amounts (e.g., 1000 USDS) require base-unit values like `1000 * 1018`, which exceed the `Int64` range ($2^{63} - 1$ or $\sim 9.22e18$).

This causes runtime errors such as `Int literal out of bounds`, blocking mint/transfer/allocate flows for common 18-decimal assets.

Similarly, the `isValidTxParam` in `StateCommitment.daml` validates that every `IntParam` is greater than 0, but due to the max value of `Int` types, transaction fails for values above $2^{63} - 1$.

RECOMMENDATION

Use `Numeric` / `Decimal` for on-ledger amounts or enforce a lower `localDecimals` ceiling (and document the max supported supply). Alternatively, scale tokens to fit `Int64` and expose that limit in client UIs.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-15**

ACKNOWLEDGED

Receiver Needs Transfer Rule Disclosure

CATEGORY Trust Assumptions

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/main/daml/Layerzero/Oft/daml/Transfer/OftTransferOffer.daml#L72>

REVIEW Main Review

DESCRIPTION

`OftTransferOffer` accepts by exercising `OftTransferRule_Execute`, which requires the `OftTransferRule` contract data. The rule is only visible to the admin (no observers), so receivers will not see it unless the admin explicitly discloses it.

If disclosure delivery is delayed or unavailable, accept/reject/withdraw can fail due to missing contract visibility, leaving offers stuck.

RECOMMENDATION

Make the transfer rule publicly discoverable (e.g., add observers, a registry, or a public fetch/disclosure flow) or document and automate disclosure to all potential receivers.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Observers Are Not Propagated In Commitment

CATEGORY Validation
LOCATION Committer.daml
REVIEW Main Review

— DESCRIPTION

Requests capture an observers list but Committer choices take a fresh `observers` argument and never read the observers stored on the Request. The owner can therefore collect a fee from a requester but omit that requester (and anyone else) from the resulting `StateCommitment`, preventing them to observe their own requests' state commitment and undermining the transparency.

— RECOMMENDATION

Pull the observer list from the Request contract when accepting/rejecting and reuse it or its superset for the resulting `StateCommitment`

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Cancelled Amulet Transfers Cause Locked Funds

CATEGORY Unexpected Behavior
LOCATION LzSendRequest.daml
REVIEW Main Review

DESCRIPTION

Users lock their OFT and must provide a valid Amulet transfer instruction, `feeCid`, to be able to send cross-chain token transfer.

Because `AmuletTransferInstruction` exposes the generic `TransferInstruction_Withdraw` choice to its sender, users can cancel their transfer instruction and get their amulet back. Doing so archives the `feeCid` and the request cannot be accepted or rejected.

There's no fallback path that unlocks the OFT when fee settlement fails, and nothing prevents users from calling `TransferInstruction_Withdraw`. As a result, a user who cancels their fee, whether intentionally or unintentionally, without understanding the potential consequences, ends up bricking their own cross-chain transfer.

RECOMMENDATION

Warn or prevent users from withdrawing Amulet instructions while a request is pending. Alternatively, consider allowing users to provide new `feeCid`s to unlock their OFTs in case of failure.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

TransferImpl bypasses blacklist

CATEGORY	Unexpected Behavior
LOCATION	TransferImpl.daml
REVIEW	Main Review

— DESCRIPTION

The core transfer routine accepts any sender and receiver without enforcing the blacklist. The only blacklist validation occurs at higher level entry points, but the shared transfer implementation itself never re-validates the parties. This creates a direct bypass path: any template or helper that calls the transfer implementation directly can move value to a blacklisted receiver even if that receiver was already blacklisted at the time of initiation.

Example scenario: Alice is added to the blacklist after prior policy violations. Bob still wants to pay Alice. An internal helper template or admin tool that calls the core transfer implementation directly constructs a transfer from Bob to Alice and executes it without any blacklist checks. The transfer succeeds and Alice receives a holding despite being blacklisted, because the only guard was at the factory entry point and that guard was bypassed.

Because the transfer implementation is reusable within the package, this is a defense in depth gap that can easily surface through new features, admin tooling, or alternative entry points. The policy becomes dependent on every caller remembering to apply blacklist checks, rather than being enforced where value actually moves.

Impact: blacklisted parties can still receive holdings when a call path reaches the core transfer logic without the factory guard, undermining compliance and governance rules.

— RECOMMENDATION

Enforce blacklist checks inside the core transfer implementation or within the execution rule that applies the holding action. Pass the blacklist set into the core transfer routine and reject any sender or receiver that appears in it. This ensures blacklist policy is enforced at the point of value movement regardless of the caller.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Non-hex compose payload accepted

CATEGORY	Validation
LOCATION	MessageEncoding.daml
REVIEW	Main Review

DESCRIPTION

The message encoder appends `composeMsg` directly to the encoded payload without validating that it is hex. When `composeMsg` is non empty, the resulting `IzMessage` contains the fixed hex fields followed by arbitrary text. This produces a malformed payload that downstream systems expecting hex encoded bytes cannot decode.

A caller can submit a cross chain send with `composeMsg` set to non hex content. The on ledger flow succeeds and creates the request, but the relay or destination decoder can reject the payload as invalid. This leaves the request in a stuck or failed state until it is explicitly rejected or expired, and creates a mismatch between on ledger intent and off chain processing.

Impact: cross chain sends can be made unprocessable by supplying non hex compose payloads, causing denial of service and operational failures. Funds may remain locked until the request is rejected or expired.

RECOMMENDATION

Validate `composeMsg` as hex before encoding, including even length and maximum size checks. Alternatively, treat `composeMsg` as raw bytes and hex encode it in the encoder so the message is always well formed.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

feeCid Reuse Can Lock Cross-Chain Requests

CATEGORY Unexpected Behavior
LOCATION OftFactory.daml, FeeUtils.daml
REVIEW Main Review

DESCRIPTION

The request factory accepts a `feeCid` and calls `validateFee` during request creation, but it does not reserve that fee instruction or enforce a one-to-one relationship between a fee and a live request. This affects both request types exposed by the factory: the cross-chain send flow (`LzSendRequest`) and the generic request flow (`Request`). Both creation paths validate the fee and then store the same `feeCid` on a newly created request without any uniqueness guard.

As a result, multiple requests can be created using the same fee instruction, including mixed reuse across request types. When any one of these requests is later finalized (accept, reject, or expire), the referenced fee is consumed through the transfer instruction interface. All remaining requests that reference the same fee then become permanently unfinalizable because fetching or exercising the fee fails.

This produces a liveness failure mode. A sender can create a cross-chain request and then create a generic request with the same fee, or create two generic requests with the same fee. Whichever request is finalized first archives the fee instruction and effectively bricks the others. In the cross-chain case, the stuck requests can also leave associated OFT holdings locked. This behavior is also contradicted by regression intent that expects fee reuse to be rejected, including

```
test_P0_FEE_feeCid_reuse_rejected_for_LzSendRequest_then_generic_Request ,  
test_P0_FEE_feeCid_reuse_rejected_for_two_generic_Requests and  
test_ILZRF_EDGE_02_fee_reuse_rejected .
```

RECOMMENDATION

Introduce an explicit reservation or uniqueness guard keyed by `feeCid` and enforce it in both request creation paths. A simple approach is to create a small reservation contract with a contract key on `feeCid` during request creation, and to archive that reservation during accept, reject, or expire. This causes fee reuse to fail early and deterministically across both request types.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/bb181be8ed348e6b30797680c94761d6>

L-05 feeCid Reuse Can Lock Cross-Chain Requests

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Update returns auth error not constant

CATEGORY	Unexpected Behavior
LOCATION	0ftTransferOffer.daml, TransferInstructionV1.daml
REVIEW	Main Review

DESCRIPTION

The `TransferInstruction` `Update` choice is controlled by the instrument admin and every party listed in `extraActors`. When a caller includes `extraActors` that do not authorize the transaction, Daml rejects the exercise at the authorization layer before the contract code runs. In this codebase, the update implementation always aborts with `ERR_OFT_UPDATE_NOT_SUPPORTED`, but that error is never reached if any `extraActor` did not sign. As a result, callers can receive a generic authorization error instead of the expected constant.

This creates inconsistent error behavior for the same unsupported operation. Systems that rely on error codes to detect unsupported updates can misclassify these failures, and tests that expect a specific error constant will fail unless all `extraActors` are also signers.

Impact: no state change occurs, but error handling is unreliable. This can confuse off ledger automation and test suites that depend on specific error codes.

RECOMMENDATION

Ensure `extraActors` are included as authorizers whenever using `Update`, or avoid specifying `extraActors` for unsupported update calls. If a consistent error surface is required, constrain callers to pass only signers as `extraActors` and document this behavior.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Fee lock mismatch bricks request expiry

CATEGORY DoS
LOCATION LzSendRequest.daml
REVIEW Main Review

DESCRIPTION

The request expiry flow can become permanently unavailable when the configured fee lock duration is shorter than the request expiration window. The `LzSendRequest` expire choice withdraws the fee as part of expiry, which calls the `Amulet` withdraw path. If the underlying fee lock has already expired or is otherwise invalid, the withdraw step fails and the entire expire transaction aborts. Because expiry is only permitted after `expirableAfter`, this creates a trap window where the request is eligible for expiry but the fee can no longer be withdrawn. In that state, accept, reject, and expire can all be blocked by process and time constraints, leaving the request unfinalizable.

A realistic example is an admin configuring `minLockDuration` to 30 seconds and `expirableAfter` to 120 seconds, then Alice creates a cross-chain send to Bob. If Bob or the `IzEndpoint` does not finalize the request, Alice attempts to expire it after 120 seconds, but the fee lock has already expired at 30 seconds, so withdraw fails and the expire transaction aborts. The request remains active, Alice's locked OFT holding stays locked and her fee stays in limbo with no recovery path. The practical impact is a liveness deadlock where locked OFT holdings and fee funds remain stuck on ledger with no valid path to resolution. This can be triggered by configuration alone without a malicious actor.

RECOMMENDATION

Enforce a configuration invariant that guarantees the fee lock remains valid through the expiry window, such as requiring `minLockDuration` to be greater than or equal to `expirableAfter` at factory creation or request creation time. Alternatively, change the expire path to handle an already expired fee lock in a controlled manner, for example by allowing expiry to complete without withdrawing the fee and emitting a dedicated error or event that records the fee recovery failure.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Local decimals mismatch not enforced

CATEGORY	Unexpected Behavior
LOCATION	OAppImpl.daml
REVIEW	Main Review

DESCRIPTION

The cross chain send path accepts a holding whose `localDecimals` do not match the factory `localDecimals` and still proceeds to encode the message and lock or burn funds. The send logic uses the factory `localDecimals` for message encoding and conversion, but it does not verify that the holding is in the same unit system. This allows a misconfigured factory or misminted holding to produce a message that represents a different numeric value than the amount actually locked or burned on the source chain. Because the receive path accepts a raw amount parameter from the off chain worker and does not derive that amount from the encoded message on ledger, the ledger trusts whatever the off chain decoder produces. That means any `localDecimals` mismatch silently shifts the encoded amount and can lead to incorrect minting on the destination chain without any on ledger correction. For example, a holding with 18 decimals sent through a factory configured for 6 decimals can encode an amount that is one trillion times larger than the true value, and the off chain worker will mint that value if it decodes the message literally. The result is cross chain accounting drift, silent inflation or deflation, and loss of supply integrity with no on ledger invariant tying the burned amount to the minted amount.

RECOMMENDATION

Enforce `holding.localDecimals == factory.localDecimals` during `OApp_LzSend` and reject the send if they differ. Consider also enforcing this invariant at holding creation or factory creation to prevent inconsistent instruments. If off chain decoding remains the source of the receive amount, add an on ledger binding such as a message hash or deterministic decode verification so that the amount supplied to `OApp_LzReceive` can be validated against the encoded message.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Redundant Holding State Check During lzSend

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/c614e08a5acc017d755dbf0866854e8545b4d34e/daml/Layerzero/Oft/daml/Common/Utils.daml#L87-L91
REVIEW	Main Review

DESCRIPTION

The `applyHoldingAction` function iterates over `allHoldings` to ensure that all holdings share the same lock state and decimals. This loop executes regardless of the number of holdings; even when only a single `inputHoldingCid` is provided, the validation redundantly compares `firstHolding` with itself. Since the `lzSend` flow always supplies exactly one input holding, this redundant check is performed on every `lzSend` action.

RECOMMENDATION

Perform the validation only when the `allHoldings` count is greater than one.

RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Empty lockers brick locked holdings

CATEGORY	Validation
LOCATION	Oft.daml, OftTransferOffer.daml
REVIEW	Main Review

DESCRIPTION

A lock with an empty lockers list creates a holding that no party can ever unlock. The unlock choice on an OFT holding is controlled by `lockHolders`, which is derived directly from the lock's lockers. If lockers is empty, the controller set is empty and there is no authorized party who can exercise `Oft_Unlock`. The holding remains locked indefinitely even though the system believes it is unlockable. The same pattern affects locked payout offers because the `ILockedPayout` view derives the `lockHolder` using `head` lockers, which fails or produces an invalid controller when lockers is empty. As a result, force accept and force reject operations become unusable, leaving locked payout offers permanently stuck. This can be triggered accidentally by misconfiguration or by any workflow that constructs a `Lock` without ensuring at least one locker.

RECOMMENDATION

Reject empty lockers at lock creation by adding an invariant that lockers must be non empty. Alternatively, if empty lockers are not expected, default the lockers list to a safe party such as the instrument admin. Also make `ILockedPayout` view logic defensive by avoiding head on an empty list and rejecting force accept or force reject when no valid `lockHolder` exists.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Fee refund context crashes on missing keys

CATEGORY	Error
LOCATION	Request.daml, LzSendRequest.daml
REVIEW	Main Review

— DESCRIPTION

The fee refund path relies on `ChoiceContext` values that are looked up with unsafe pattern matches. In `FeeUtils.acceptFeeWithRefund`, the code assumes that amulet-rules and open-round keys exist and are of the correct type and it deconstructs them with a `let Some` pattern. If either key is missing or has the wrong type, the lookup returns `None` and the pattern match throws a runtime exception. This code path is exercised by reject with refund flows that call `acceptFeeWithRefund`, such as `Request_Reject` and `LzSendRequest_Reject`. The failure mode is an opaque crash rather than a controlled error code, which makes integration debugging difficult and can lead to unexpected refund failures in production. The unsafe lookup logic is shown below.

```
let Some (AV_ContractId amuletRulesCidAny) = TM.lookup "amulet-rules" amuletContext.values
let amuletRulesCid : ContractId AmuletRules = coerceContractId amuletRulesCidAny
let Some (AV_ContractId openRoundCidAny) = TM.lookup "open-round" amuletContext.values
let openRoundCid : ContractId OpenMiningRound = coerceContractId openRoundCidAny
```

— RECOMMENDATION

Replace the unsafe pattern matches with explicit validation and `assertMsg` errors for missing or malformed context entries. Add dedicated error constants for missing amulet-rules and missing open-round and for wrong value types, so integration clients receive deterministic failures instead of runtime exceptions.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Missing requestedAt <= now in Allocation

CATEGORY Validation
LOCATION OftFactory.daml
REVIEW Main Review

— DESCRIPTION

The `allocationFactory_allocateImpl` function does not validate that the `requestedAt` parameter is in the past or present, unlike the `transferFactory_transferImpl` function which performs this check. It only validates: `allocation.settlement.requestedAt <= requestedAt`.

This causes inconsistency with transfer function validation pattern.

— RECOMMENDATION

Consider adding `requestedAt <= now` validation to ensure that `allocation.settlement.requestedAt <= requestedAt <= now`.

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Missing Validations When Updating Factory Params

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/c614e08a5acc017d755dbf0866854e8545b4d34e/daml/Layerzero/Oft/daml/OftFactory.daml#L240-L255>

REVIEW Main Review

DESCRIPTION

`SetRequestFactory` and `SetTransferRule` setters simply archive the current `OftFactory` and recreate it with the new contract IDs without performing any checks and blindly updates with admin provided values.

A misconfiguration or compromised issuer can point the OFT to an `ILzRequestFactory` owned by a different party rather than `LzEndpoint` or to a transfer rule that doesn't point to the same `instrumentId`.

RECOMMENDATION

Add validations to setters to ensure `newRequestFactoryCid` is also owned by the `LzEndpoint` and `newTransferRuleCid` targets the same `instrumentId`.

RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Missing Replay Protection For 0App_LzReceive

CATEGORY	Trust Assumptions
LOCATION	https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/main/daml/Layerzero/0App/daml/I0App.daml#L62
REVIEW	Main Review

DESCRIPTION

`0App_LzReceive` mints based solely on the provided `receiver` and `amount`, with no on-ledger check for `(origin, guid)` uniqueness. The TypeScript endpoint library includes replay protection (verify → `IzReceive` with inbound payload hashes), but the HTTP controllers in this repo are TODO/501 and do not wire in those checks. If the endpoint submits the same message twice, the ledger will mint twice.

RECOMMENDATION

Add on-ledger replay protection (store processed GUIDs or nonces per origin) or enforce the `verify→IzReceive` flow in the endpoint service before submitting the Daml command. Document this as a trust assumption if left off-ledger.

RESOLUTION

Status: Acknowledged.

LOW **L-16**

ACKNOWLEDGED

Request factory accepts mismatched IOApp

CATEGORY DoS
LOCATION LzRequestFactory.daml
REVIEW Main Review

DESCRIPTION

The request factory allows `LzSendRequest` creation with an arbitrary `IOApp` contract ID and does not verify that the referenced `IOApp` belongs to the same endpoint that owns the factory. As a result, a sender and issuer can create a request under one endpoint while pointing the callback target at a different `IOApp` controlled by another endpoint. The request is successfully created and the sender's holding is locked, but the later accept, reject, or expire flow calls the `IOApp` callback under the request factory owner. That callback requires authorization from the `IOApp`'s own endpoint, so the transaction fails and the request becomes unfinalizable.

A realistic scenario is a user or integrator submitting direct `LzSendRequest_Create` calls through an automated issuer signer. Alice submits a request via the endpoint A factory but supplies the `IOApp` from endpoint B. The issuer co-signs automatically, the request is created and Alice's holding is locked. When endpoint A attempts to accept or expire the request, the callback to the endpoint B `IOApp` cannot be authorized by endpoint A and the transaction aborts. Alice's funds and fee remain locked with no resolution path unless endpoint B cooperates and the endpoint operator sees repeated failures for a request that should have been rejected at creation.

RECOMMENDATION

At request creation, validate that the provided `IOApp` belongs to the same endpoint as the request factory by fetching its view and comparing the `lzEndpoint` to the factory owner. Additionally, validate that the `IOApp` matches the instrument implied by the holding or derive the `IOApp` from a keyed factory lookup instead of accepting it as a free parameter. Reject creation if these checks fail to prevent unfinalizable requests.

RESOLUTION

Status: Acknowledged.

LOW **L-17**

ACKNOWLEDGED

LzSend accepts negative dstEid values

CATEGORY Validation
LOCATION 0AppImpl.daml
REVIEW Main Review

DESCRIPTION

`dstEid` is an `Int` and is passed directly from `SendParam` into `LzSendRequest` without validation. Negative or invalid EID values can be used to create requests that will never be processed off-ledger.

RECOMMENDATION

Validate `dstEid` as a non-negative, supported EID at `0App_LzSend` time (or in the request factory) and reject invalid values early.

RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

LzSend accepts unbounded extraOptions size

CATEGORY	Validation
LOCATION	0AppImpl.daml
REVIEW	Main Review

DESCRIPTION

`sendParam.extraOptions` is stored without any length checks. A sender can submit arbitrarily large option payloads, which can bloat requests and stress off-ledger processing. The current TypeScript implementation only parses these bytes (no dynamic code execution), so the realistic risk is resource exhaustion or parsing failures that stall processing rather than code execution.

Additionally, `Request.payload` (the `[Text]` payload stored on Request) is not bounded: callers can supply arbitrarily many elements or very large text elements, producing the same bloat and off-ledger processing risks.

RECOMMENDATION

Enforce a maximum `extraOptions` length at `0App_LzSend` time (or in the request factory) and reject oversized inputs. For `sendParam.extraOptions` enforce a max byte length; for `Request.payload` enforce limits on both the number of elements and the max length per element (or a total combined size).

RESOLUTION

Status: Acknowledged.

LOW **L-19**

ACKNOWLEDGED

No Cap or Refund On Fee Payments

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/c614e08a5acc017d755dbf0866854e8545b4d34e/daml/Layerzero/Request/daml/FeeUtils.daml#L121-L125>

REVIEW Main Review

DESCRIPTION

`validateFee` only checks that amount is greater than or equal to `minFee`. When the factory owner accepts a request the code blindly exercises `acceptFee`, transferring the entire locked amount to the owner. There is no maximum bound and no refund path on accept.

A sender who mistakenly locks more Amulet than required (or is tricked by a malicious UI) permanently loses the excess once the admin accepts, even though the protocol only intended to charge the minimum fee. The only mechanism that can return funds is the reject flow (`acceptFeeWithRefund`), so any "successful" request becomes an all-or-nothing payment.

RECOMMENDATION

Consider introducing a `maxFee` parameter and enforcing `minFee <= amount <= maxFee` in `validateFee`, and/or automatically refunding the excess during acceptance

RESOLUTION

Status: Acknowledged.

LOW **L-20**

ACKNOWLEDGED

Expired fee not validated at request creation

CATEGORY Validation
LOCATION FeeUtils.daml
REVIEW Main Review

— DESCRIPTION

The `validateFee` helper checks fee sender, minimum amount, and lock duration, but does not enforce the fee transfer's `executeBefore`.

As a result, requests can be created with an already-expired fee instruction. Later, `TransferInstruction_Accept` always fails in Amulet (`assertWithinDeadline`), so accept/reject cannot proceed. Expire can still succeed once `expirableAfter` passes if the caller provides a valid `expire-lock` /open-round context; until then the request is effectively stuck.

```
-- daml/Layerzero/Request/daml/FeeUtils.daml
validateFee feeCid expectedSender minFee minLockDuration = do
  feeInstruction <- fetch feeCid
  assertMsg errRequestFactory_FeeSenderMismatch $ feeInstruction.transfer.sender ==
    expectedSender
  assertMsg errRequestFactory_InsufficientFee $ feeInstruction.transfer.amount >= minFee
```

— RECOMMENDATION

Add an `executeBefore` check when validating the fee instruction (e.g., require `executeBefore > now` or `now <= executeBefore`), or reject request creation when the fee is already expired.

PROOF OF CONCEPT <https://gist.github.com/wafflemakr/a3ce35e48f07d3d063e0067655cb269f>

— RESOLUTION

Status: Acknowledged.

LOW **L-21**

ACKNOWLEDGED

stateRoot accept arbitrary Text not only bytes32

CATEGORY	Validation
LOCATION	StateCommitment.daml
REVIEW	Main Review

— DESCRIPTION

`stateRoot` is declared as Text and is not validated for hex format or fixed length. A committer can store arbitrary or malformed roots. Off-ledger verifiers expecting a bytes32 hex value may fail to parse or silently accept invalid data, breaking integrity guarantees.

— RECOMMENDATION

Validate `stateRoot` as hex and enforce exact length (64 hex chars = bytes32) after stripping `0x`. Optionally validate signature formats at the same boundary.

— RESOLUTION

Status: Acknowledged.

LOW **L-22**

ACKNOWLEDGED

requestFactoryCid Becomes Stale on Upgrade

CATEGORY	DoS
LOCATION	Committer.daml
REVIEW	Main Review

DESCRIPTION

The Committer contract stores a reference to a specific `ILzRequestFactory` contract via `requestFactoryCid`. If the referenced factory contract is archived (for example, calling `setMinFee`), all choices in Committer that rely on exercising `requestFactoryCid` (such as `AcceptRequest` and `RejectRequest`) will fail with a "contract not active" error. This tightly couples the Committer to a single, immutable factory instance, causing all existing Committer contracts to become unusable.

Impact:

- All Committer contracts referencing an archived factory become permanently unusable for request processing, this requires a new commitment contract

RECOMMENDATION

Refactor Committer to avoid storing a static `ContractId` for the factory. Instead, dynamically look up the current active factory contract when processing requests, or consider adding consuming choice to update the `requestFactoryCid`

RESOLUTION

Status: Acknowledged.

LOW **L-23**

ACKNOWLEDGED

AllocationFactory allocate needs disclosure

CATEGORY Unexpected Behavior
 LOCATION OftFactory.daml
 REVIEW Main Review

— DESCRIPTION

Several user controlled entry points are hosted on the factory contract, but the factory is not visible to those users by default. The factory is signed by the instrument admin and observed by the endpoint, while the relevant choices are controlled by the sender. Under Canton privacy rules, the controller must be able to see the contract that hosts the choice, so these flows depend on off ledger disclosure for basic availability. This affects `TransferFactory_Transfer`, `AllocationFactory_Allocate` and `OApp_LzSend`, all of which are implemented on the same factory contract even though their controllers are user parties.

The visibility mismatch is structural. The factory that hosts these choices does not list the expected caller as an observer, but the choices are controlled by that caller.

```
template OftFactory
with instrumentId : InstrumentId; lzEndpoint : Party
where
signatory instrumentId.admin
observer lzEndpoint
interface instance TransferFactory for OftFactory where
transferFactory_transferImpl _self TransferFactory_Transfer{...} = do ...
interface instance AllocationFactory for OftFactory where
allocationFactory_allocateImpl _self AllocationFactory_Allocate{...} = do ...
interface instance IOApp for OftFactory where
oApp_lzSendImpl _ OApp_LzSend{...} = do ...
```

If disclosure delivery is missing, delayed, or scoped incorrectly, these user entry points fail with visibility errors even when the user is the controller. This creates a consistent liveness dependency on disclosure infrastructure rather than on ledger observers and signatories, and it is likely to surface as intermittent production failures.

— RECOMMENDATION

Make the factory visible to parties who are expected to exercise these user entry points before they need to use them. A robust approach is to introduce a sender visible registry

L-23 AllocationFactory allocate needs disclosure

CONTINUED

or delegation contract that references the current factory and can exercise these choices on behalf of the sender. If off ledger disclosure remains the intended approach, treat factory disclosure to users as a required provisioning step and add monitoring that detects missing disclosure before user flows are invoked.

— RESOLUTION

Status: Acknowledged.

LOW **L-24**

ACKNOWLEDGED

Missing Time Validations In Allocation Creation

CATEGORY Validation
LOCATION OftFactory.daml
REVIEW Main Review

DESCRIPTION

`allocationFactory_allocateImpl` compares `allocation.settlement.requestedAt` only against the caller-provided `requestedAt`, and never checks `allocateBefore` / `settleBefore` against `getTime`. This allows allocations to be created after their own `allocateBefore` deadline (or with future `requestedAt`), weakening time-bound guarantees.

RECOMMENDATION

Use `getTime` to validate `requestedAt` and require `allocation.settlement.allocateBefore > now` (and optionally `settleBefore > now`) at allocation creation.

PROOF OF CONCEPT <https://gist.github.com/wafflemakr/764fef17d127a9d5322265835c6f69ac>

RESOLUTION

Status: Acknowledged.

LOW **L-25**

ACKNOWLEDGED

Missing Validations When Creating Offers

CATEGORY Validation
LOCATION OftTransferOffer.daml
REVIEW Main Review

DESCRIPTION

`OftTransferOffer` is missing the following validations:

- `transfer.inputHoldingCids` uniqueness. If an offer is created with duplicate holdings (possible via direct template creation by issuer+sender), accept/reject/withdraw will attempt to process the same locked holding twice, causing the transaction to fail and leaving the offer pending with locked funds. In normal TransferFactory flow, duplicates are rejected earlier, so this is primarily a footgun for direct creation paths.
- Locked `transfer.inputHoldingCids`. If an offer is created with unlocked holdings (possible via direct template creation by issuer+sender), accept/reject/withdraw will all fail because `UnlockMergeSplitTransfer` and `Oft_Unlock` require locked inputs. The offer remains pending with no valid resolution path.

RECOMMENDATION

Add a uniqueness check for `transfer.inputHoldingCids` in `OftTransferOffer` (or in the transfer rule) to fail fast on malformed offers.

Additionally, add a lock-state check for `transfer.inputHoldingCids` at offer creation (or in the transfer rule) so malformed offers fail fast.

PROOF OF CONCEPT <https://gist.github.com/wafflemakr/60c41cb75ce741c10ae854bf40900a10>

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Misleading Comment In createPayoutHoldings

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/c614e08a5acc017d755dbf0866854e8545b4d34e/daml/Layerzero/Oft/daml/OApp/OAppImpl.daml#L80
REVIEW	Main Review

DESCRIPTION

The comment on `createPayoutHoldings` states "returning any remaining change to the sender as an unlocked holding," but the function actually returns only the newly created locked payout holdings. Unlocked remaining change is minted later in the flow, not in this function.

RECOMMENDATION

Update the comment on `createPayoutHoldings` function

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Endpoint Can Exercise Callback Choices

CATEGORY Trust Assumptions

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/main/daml/Layerzero/0App/daml/I0App.daml#L80-L108>

REVIEW Main Review

DESCRIPTION

`0App_OnLzSendAcceptedCallback/Rejected/Expired` are directly callable by `lzEndpoint` and do not verify that the holding/payouts belong to an active `LzSendRequest`.

RECOMMENDATION

Bind callbacks to a specific `LzSendRequest` or validate the holding/payouts against request state before mutating.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Duplicate payout receivers not coalesced

CATEGORY	Unexpected Behavior
LOCATION	0AppImpl.daml
REVIEW	Main Review

DESCRIPTION

The payout receiver list for cross-chain sends is used as-is when creating payout holdings and offers. If the sender supplies the same receiver multiple times, the system creates multiple payout holdings and multiple transfer offers for that receiver instead of a single aggregated payout. A regression test expects duplicate receivers to be coalesced into one offer, but the current behavior generates one offer per list entry.

A realistic scenario is Alice initiating a cross-chain send and specifying a service provider list that contains the same receiver twice, for example Carol with two separate amounts. The system creates two locked payout holdings and two payout offers for Carol. When the endpoint later forces acceptance, Carol receives the total amount across two offers rather than a single consolidated payout. This does not break conservation of value, but it complicates accounting, reconciliation and downstream monitoring that assumes one payout per receiver.

RECOMMENDATION

Consider normalizing the receiver list before creating payout holdings and offers. Consider also aggregating amounts by receiver so each receiver appears at most once, drop zero-amount entries and preserve the original total payout amount.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

ACKNOWLEDGED

Missing issuer auth on OApp LzSend

CATEGORY	Validation
LOCATION	IOApp.daml, OftFactory.daml, OAppImpl.daml
REVIEW	Main Review

DESCRIPTION

OApp `LzSend` is controlled solely by the sender, but the request factory interface for creating a cross-chain send requires both sender and issuer authorization. The OApp flow constructs the request with issuer set to the token admin, yet it does not require the admin to be a submitter. As a result, the transaction succeeds even when the issuer is absent from the submitter set, which bypasses the intended issuer co-sign requirement.

A realistic scenario is Alice holding OFT tokens for a regulated instrument. The issuer expects to approve every cross-chain transfer by co-signing the request. Alice calls OApp `LzSend` on the factory using only her own party and the `LzEndpoint` as submitters. The system locks her holding, splits payouts and creates an `LzSendRequest` that records the issuer as the admin, even though the admin never authorized the transaction. Off-chain relayers that watch for requests and trust the issuer field now see a seemingly authorized request and proceed with cross-chain delivery. The issuer loses control over outbound cross-chain transfers and the request queue can be flooded with unauthorized requests, creating operational risk. This goes against any issuer control and any compliance policy that requires issuer review.

This mismatch also breaks rollback expectations. A missing issuer authorization should cause the request creation step to fail and roll back all intermediate artifacts, but the system allows the transaction to complete. In practice, any token holder can initiate a cross-chain transfer that off-chain relayers will treat as issuer approved, because the request records the issuer/admin even though they did not sign. It also enables request flooding that the issuer or relayer must triage, creating an operational risk and potentially allowing unauthorized transfers to execute on the destination chain.

RECOMMENDATION

Make issuer authorization explicit and enforce it at the entry point. Require the issuer/admin to co-sign OApp `LzSend`, or move request creation into a choice that is directly controlled by the issuer so that missing issuer authorization fails before any side effects occur. Keep the interface contract and the implementation consistent so that the stated

I-04 Missing issuer auth on OApp LzSend

CONTINUED

sender plus issuer policy is actually enforced. Add or keep a regression test that submits OApp LzSend without the issuer and verifies it fails with an authorization error and rolls back all artifacts.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Cannot burn locked holdings in BurnMint

CATEGORY	Validation
LOCATION	BurnMintImpl.daml
REVIEW	Main Review

DESCRIPTION

The `executeBurnMint` function allows the admin to burn input holdings and mint new holdings for specified outputs. In addition to that, only checks blacklist for output owners, not input holding owners, allowing admin to burn holdings of blacklisted parties.

However, it requires all input holdings to be unlocked (`mustBeUnlocked oft.lock`).

This can prevent burning locked holdings, even when the admin has authorization to burn them.

Example Scenario

1. Admin creates Oft manually for a party (bypasses factory validation)
2. Admin locks the Oft with `lockers = [party]` (admin is NOT a locker)
3. Party gets blacklisted
4. Admin cannot burn the locked holding because:
 - `mustBeUnlocked` check fails (holding is locked)
 - Admin cannot unlock it (admin is not in lockers list)

RECOMMENDATION

Consider allowing admin to burn locked holdings for blacklisted parties.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

No validation on SetExpirableAfter

CATEGORY Best Practices
LOCATION LzRequestFactory.daml
REVIEW Main Review

— DESCRIPTION

There are no checks in `SetExpirableAfter`, allowing arbitrary values to be set, including negative, and excessively large values.

```
-- Admin sets negative expirableAfter (born expired)
exerciseCmd requestFactoryCid LzRequestFactory.SetExpirableAfter with
newExpirableAfter = seconds (-3600) -- -1 hour
```

Setting Excessively large values:

```
-- Admin sets expirableAfter to 1000 years
exerciseCmd requestFactoryCid LzRequestFactory.SetExpirableAfter with
newExpirableAfter = days 365000 -- 1000 years
```

— RECOMMENDATION

Add validation in the `SetExpirableAfter` choice to ensure it is always positive and within specific bounds.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

No validation on SetMinLockDuration

CATEGORY Best Practices
LOCATION LzRequestFactory.daml
REVIEW Main Review

DESCRIPTION

There are no checks in `SetMinLockDuration`, allowing arbitrary values to be set, including negative ones.

```
-- Admin (maliciously or accidentally) sets a negative minLockDuration  
exerciseCmd requestFactoryCid LzRequestFactory.SetMinLockDuration with  
newMinLockDuration = seconds (-3600) -- -1 hour
```

Additionally, there is no validation preventing excessively large values.

```
-- Admin sets minLockDuration to 100 years  
exerciseCmd requestFactoryCid LzRequestFactory.SetMinLockDuration with  
newMinLockDuration = days 36500 -- 100
```

RECOMMENDATION

Ensure that `minLockDuration` is always positive and within specific bounds.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

DoS Window on New Cross-Chain Sends

CATEGORY Upgradeability
 LOCATION LzRequestFactory.daml , OftFactory.daml
 REVIEW Main Review

DESCRIPTION

The `OftFactory` stores a reference to `LzRequestFactory`:

```
template OftFactory
with
  requestFactoryCid : ContractId ILzRequestFactory -- STORED reference
...
```

When users initiate cross-chain sends, this reference is passed to `LzSend`:

```
-- OftFactory.daml:205
LzSend instrumentId localDecimals sharedDecimals issuerFeeBps
transferRuleCid requestFactoryCid lzEndpoint sender sendParam
oftReceivers oftReceiverAmounts feeCid oAppCid
```

Which then exercises it to create the request:

```
-- OAppImpl.daml:210
exercise requestFactoryCid ILzRequestFactory.LzSendRequest_Create with
sender
issuer = instrumentId.admin
...
```

Problem: When admin updates `LzRequestFactory` via consuming choices:

- `SetMinFee`
- `SetMinLockDuration`
- `SetExpirableAfter`

The old factory is archived, but `OftFactory` contracts still reference the old `requestFactoryCid`. So any `OApp_LzSend` request sent by the user will fail cause we are calling archived contract.

I-08 DoS Window on New Cross-Chain Sends

CONTINUED

RECOMMENDATION

Ensure that `SetRequestFactory` is called immediately after any of these functions are invoked.:

- `SetMinFee`
- `SetMinLockDuration`
- `SetExpirableAfter`

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-09**

ACKNOWLEDGED

Unused Errors

CATEGORY	Best Practices
LOCATION	Errors.daml
REVIEW	Main Review

DESCRIPTION

Multiple errors are unused in the codebase:

- `errLzSend_NoOutput`
- `errOft_SenderBlacklisted`
- `errOft_ReceiverBlacklisted`
- `errOftTransfer_SenderReceiverSame`
- `errOftAllocation_InstrumentMismatch`
- `errOftAllocation_AmountNotPositive`
- `errLzSendRequest_UnauthorizedReject`
- `errLzSendRequest_RejectTooEarly`
- `errLzSendRequest_UnauthorizedExpire`
- `errOftTransfer_InstrumentMismatch`

RECOMMENDATION

Consider removing the unused errors.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Sender Visibility Inconsistency in Request Event

CATEGORY	Best Practices
LOCATION	Request.daml
REVIEW	Main Review

DESCRIPTION

In `lzSendRequest.daml`, the emitted event templates explicitly include the sender as an observer, so the user always has visibility.

In contrast, the event templates in `Request.daml` (`RequestAcceptedEvent`, `RequestRejectedEvent`, `RequestExpiredEvent`) only use the observers list provided at request creation time and do not automatically add the sender as an observer.

This results in inconsistent behavior compared to `lzSendRequest.daml`.

RECOMMENDATION

Document this requirement clearly: the request sender must include themselves in the observers list when creating a Request.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Transfer rule upgrade can brick pending requests

CATEGORY Upgradeability

LOCATION <https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/main/daml/Layerzero/Oft/daml/OftFactory.daml#L249>

REVIEW Main Review

DESCRIPTION

`OftTransferOffer` stores a concrete `transferRuleCid` and always exercises that rule on accept/reject. If the admin upgrades the transfer rule and archives the old rule contract, any pending offers that reference the old rule become unprocessable (accept/reject/withdraw fail). This is a liveness hazard during upgrades.

RECOMMENDATION

Keep old rules alive until outstanding offers are resolved, or route offers through a stable registry/key so they always resolve to the current rule. Alternatively, implement an upgrade-safe indirection or migrate outstanding offers.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-12

ACKNOWLEDGED

Developed testing suite

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/audit_tests/daml/test_validation_spec/T
REVIEW	Main Review

— DESCRIPTION

Testing suite link: https://github.com/GuardianOrg/canton-ver-team1-1768236800046/blob/audit_tests/daml/test_validation_spec/TESTS_BY_CONTRACT.md

This entry records the test validation work performed in the `daml/test_validation_spec` suite. New test modules were implemented to cover additional edge cases, interface bypass attempts, cross-factory isolation, request and fee lifecycle integrity and `IzReceive` validation and the suite was executed to generate transaction traces and confirm behavioral outcomes. The `TESTS_BY_CONTRACT` index was regenerated from the latest JUnit output and augmented with human readable descriptions derived from source comments to ensure the catalog reflects the current test inventory and pass or fail status. Full suite execution produced transaction logs and summaries that were used to correlate failing tests with expected policy gaps and behavior mismatches. The work provides a traceable record of what was tested, how it was run and which areas are intentionally failing due to missing validations.

The full suite was executed with a transactions output path and JUnit output to preserve an auditable trail of test results and traces.

```
daml test --package-root daml/test_validation_spec --transactions-output
daml/test_validation_spec/tx_logs/latest --junit
daml/test_validation_spec/daml_test_validation_spec_latest.xml
```

— RECOMMENDATION

No recommendation. Informational record only.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.